一种基于单密钥同态加密的高效隐私增强联合学习算法

An Efficient Privacy-Enhanced Federated Learning With Single-Key Homomorphic Encryption

Yilong Liu, Hong Zhang, Miao Wang, Qiqi Xie, *Student Member, IEEE*, and Liqiang Wang

***Abstract*—As the proliferation of Internet of Things (IoT) devices continues, vast amounts of data are being collected on various end devices. ~~However,~~ uploading these data to the cloud for centralized processing poses significant privacy risks. Federated learning (FL) addresses this issue by sharing model updates instead of raw data, which helps mitigate privacy concerns. ~~Nonetheless~~, model updates transmitted in plaintext remain vulnerable to inference and reconstruction attacks. While homomorphic encryption (HE) can enhance security, traditional single-key schemes struggle to defend against collusion. Multikey HE schemes introduce substantial computational and communication overhead, making them impractical for resource-constrained IoT environments. ~~In this article,~~ we propose a novel FL framework that integrates single-key HE, elliptic curve cryptography (ECC), and trusted execution environments (TEE) to achieve robust protection against collusion attacks. ~~Specifically~~, we utilize ECC-based public key encryption to secure HE private key and employ secret sharing to split the ECC private key into multiple subsecrets, which are distributed to edge nodes, ensuring no single party can independently decrypt model updates. ~~Additionally~~, we delegate the HE private key reconstruction and global model decryption processes to the TEE, and introduce a hash verification mechanism to ensure that only aggregated global model updates can be decrypted. ~~Finally~~, we provide comprehensive security proofs and extensive experimental results, demonstrating the effectiveness and superiority of the proposed framework.**

***Index Terms*—Elliptic curve cryptography (ECC), federated learning (FL), homomorphic encryption (HE), privacy preserving, trusted execution environment.**

I. Introduction

THE rapid proliferation of Internet of Things (IoT) devices has led to an unprecedented surge in data generation across various applications [1], including smart cities [2], healthcare [3], and autonomous vehicles [4]. Traditionally, these data would be transmitted to centralized cloud servers for processing. However, this centralized approach raises significant privacy concerns, as sensitive IoT data face potential exposure during cloud transmission and storage. Federated learning (FL) [5] offers a solution to this issue by enabling a decentralized model training approach. Through FL, devices collaborate by sharing model updates instead of raw data, improving data privacy protection.

~~Nonetheless,~~ recent studies have shown that even with FL, model updates transmitted in plaintext can still lead to privacy leaks [6], [7]. To address this vulnerability, various privacy-preserving techniques, such as secure multiparty computation (SMC) [8], [9], differential privacy (DP) [10], [11], and homomorphic encryption (HE) [12], [13], have been applied to FL. While SMC establishes secure protocols for aggregation, it incurs high-communication overhead. DP introduces noise to model updates, but this noise can accumulate and degrade the accuracy of the global model. HE, which allows computation on encrypted data, supports aggregation over ciphertexts without compromising model accuracy, offering a promising end-to-end encryption approach to mitigate privacy risks. ~~However~~, traditional single-key HE schemes are vulnerable to privacy leaks in multiparty environments, while multikey HE introduces substantial computational and communication costs, making it impractical in resource-constrained IoT scenarios. ~~Consequently~~, HE-based privacy-preserving FL still faces challenges in privacy protection, efficiency, and robustness.

First, conventional HE systems operate on a single-key scheme, where all devices encrypt with the same public key and share the decryption key across devices or store it on the server. This setup exposes model updates to semi-honest parties and fails to defend against collusion between devices and the server [12], [14].

~~Second,~~ although multikey HE can address these vulnerabilities, it significantly increases computational and communication costs. Multikey HE requires a complex key generation (KeyGen) process, which greatly increases communication expenses and computational complexity [13]. ~~Furthermore~~, existing multikey HE schemes are limited to simple additive aggregation [15], making them unsuitable for practical IoT applications. ~~Moreover~~, the critical issue is that multikey HE schemes are intolerant of device offline scenarios. If a single device disconnects, the entire system becomes incapable of fully decrypting the global model [15], [16].

~~Therefore~~, a multiparty secure FL framework based on single-key HE is indispensable in IoT environments. ~~To address these challenges~~, we propose a novel and efficient FL framework that combines single-key HE, elliptic curve cryptography (ECC), and trusted execution environments (TEE),

Received 7 February 2025; revised 6 March 2025; accepted 15 March 2025. Date of publication 20 March 2025; date of current version 27 June 2025. This work was supported in part by the Natural Science Foundation of Hebei Province of China under Grant F2019201361, and in part by the Innovation Capacity Enhancement Program-Science and Technology Platform Project of Hebei Province under Grant 22567638H. *(Corresponding author: Hong Zhang.)*

Yilong Liu, Hong Zhang, Miao Wang, and Qiqi Xie are with the School of Cyber Security and Computer, Hebei University, Baoding 071032, China (e-mail: liuyilongayy@gmail.com; hzhang@hbu.edu.cn; wm@hbu.edu.cn; 20227019042@stumail.hbu.edu.cn).

Liqiang Wang is with the Department of Computer Science, University of Central Florida, Orlando, FL 32816 USA (e-mail: liqiang.wang@ucf.edu).

Digital Object Identifier 10.1109/JIOT.2025.3553134

2327-4662 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Anhui University. Downloaded on March 24,2026 at 04:23:30 UTC from IEEE Xplore. Restrictions apply.

providing robust protection against collusion attacks while maintaining high efficiency. In our framework, ECC public key is employed to encrypt the HE private key, which is then divided into multiple subsecrets via secret sharing and distributed to edge nodes (ENs), ensuring no single party has independent access to the HE private key. Furthermore, the HE private key reconstruction and global model decryption are performed within a TEE, with a hash verification mechanism ensuring only aggregated global models are decrypted within the TEE. This setup mitigates risks of key exposure and unauthorized decryption requests. The contributions of this article are summarized as follows.

1) We present a novel privacy-preserving FL framework that employs an innovative key management scheme, enabling single-key HE to be securely applied in multiparty IoT environments. This framework can greatly enhance the security of FL with single-key HE.
2) We propose an advanced key management scheme using ECC public key to encrypt the HE private key, which distributes ECC subsecrets to ENs via secret sharing to defend against collusion attacks in FL.
3) We also design a novel ECC decryption algorithm based on secret sharing. This algorithm enables decryption without the need to explicitly reconstruct the private key of ECC. To ensure the security of the aggregation result decryption process, all decryption operations are performed within a TEE. Additionally, we introduce a hash verification mechanism within the TEE to prevent untrusted nodes from misusing the TEE's decryption privileges.
4) Extensive security analysis and experiments demonstrate the proposed framework's effectiveness and superiority in privacy protection and efficiency.

The remainder of this article is structured as follows. Section II reviews related work on privacy-preserving FL. Section III introduces the necessary preliminaries. Section IV presents the models and design goals. In Section V, we describe the details of our proposed scheme. Section VI provides the security analysis. Section VII covers experiments and evaluations. Finally, in Section VIII, we conclude this article and discuss future work.

II. Related Work

In this section, we review existing work on privacy-preserving FL from three perspectives: 1) SMC; 2) DP; and 3) HE. These works are compared and summarized in Table I.

TABLE I
COMPARISONS OF EXISTING WORK

Scheme	PT	CR	DR	NLMA	CCO
[17]	SMC	✓	✗	✓	H
[18]	SMC	✓	✓	✗	M
[19]	SMC	✓	✓	✓	H
[20], [21]	SMC	✓	✓	✓	H
[22], [23]	SMC	✓	✓	✓	M
[24], [25]	SMC	✓	✓	✓	M
[26]	DP	✗	✓	✗	L
[27]	DP	✗	✓	✗	L
[28], [29]	DP	✗	✓	✗	L
[30], [31], [32]	DP	✗	✓	✗	L
[33], [34], [35]	SKHE	✗	✓	✓	M
[36], [37], [38]	SKHE	✗	✓	✓	L
[14]	SKHE	✗	✓	✗	L
[15]	MKHE	✓	✗	✗	H
[39], [40]	MKHE	✓	✓	✗	M
[41]	MKHE, TEE	✓	✓	✗	M
[42]	MKHE	✓	✗	✓	M
Ours	SKHE, TEE	✓	✓	✓	L

Notes: PT: Privacy Technique; CR: Collusion Resistant; DR: Dropout Resilience; NLMA: No Loss in Model Accuracy; CCO: Computation and Communication Overhead; SKHE: Single-Key HE; MKHE: Multi-Key HE; ✓: Support; ✗: Not Support; H: High; M: Medium; L: Low.

A. SMC-Based Federated Learning

SMC enables multiple parties to jointly compute the result of a function without revealing their individual inputs. Bonawitz et al. [17] pioneered a dual-masking technique, which was combined with secret sharing, to design a secure aggregation protocol. However, this protocol required each device to communicate with the other $N-1$ devices, resulting in significant communication overhead. Bell et al. [18] optimized this approach [17] by limiting the number of neighboring devices for each device, reducing communication to only $\log(N)$ devices. This optimization required additional data structures and increased storage costs. Building on Bonawitz's work [17], Xu et al. [19] introduced a framework that combined dual-masking with homomorphic hash functions to further address integrity issues in secure aggregation. Similarly, VeriFL [20], VerSA [21], and WVFL [22] have contributed solutions to address this challenge. However, these approaches still face high computational and communication costs. To reduce the overhead of secure aggregation protocols, the FastSecAgg [23] scheme was developed, which employed multisecret sharing optimized by fast Fourier transformation. Chen et al. [24] leveraged compressed sensing techniques to reduce the transmission of local gradients, thereby lowering the cost of secure aggregation protocols. Additionally, they proposed a SMC algorithm capable of verifying participant identities. Zhang et al. [25] introduced a sketch-based FL algorithm, which further reduced the cost of secure protocols by incorporating singular value decomposition. Unfortunately, all of these approaches require multiple rounds of communication, which is impractical in IoT environments with complex network conditions.

B. DP-Based Federated Learning

DP protects privacy by adding noise to private data. Wu et al. [26] utilized local DP by adding noise to local gradients to protect participant privacy. Stevens et al. [27] proposed a novel DP-based secure aggregation scheme for learning with errors (LWE), effectively reducing computational costs through distributed discrete Gaussian mechanisms and gradient clipping. Additionally, to improve model utility under DP, innovative solutions have emerged. Asoodeh et al. [28] interpreted FL algorithms as a combination of multiple Markov kernels and calculated DP parameters based on the kernels' contraction coefficients. Cheng et al. [29] significantly improved model utility by incorporating local regularization and sparsification methods while preserving DP. Yue et al. [30]

Authorized licensed use limited to: Anhui University. Downloaded on March 24,2026 at 04:23:30 UTC from IEEE Xplore. Restrictions apply.

designed an adaptive local DP (LDP) approach using a noise scaler, which dynamically optimized the noise level for each client. Furthermore, an increasing number of schemes focus on addressing privacy budget issues in heterogeneous scenarios. Ling et al. [31] proposed an efficient heterogeneous DP method that employed a secure differentially private stochastic gradient descent (SGD) protocol combined with directional gradient histogram feature extraction. Clients with higher noise levels were assigned lower privacy budget weights to mitigate their negative impact on the aggregated model. FedMPS [32] leveraged local model partitioning and sparsification to reduce the impact of noise on model utility in heterogeneous data scenarios. However, these methods still face the challenge of balancing privacy protection and model accuracy.

C. HE-Based Federated Learning

HE offers a powerful approach to privacy protection in FL by enabling computations on encrypted data, giving rise to numerous HE-based privacy-preserving FL schemes. Phong et al. [33] employed the Paillier additive HE system to ensure privacy, though it incurred high-computational overhead. To improve efficiency, Zhang et al. [34] proposed a Paillier-based HE approach optimized by the Chinese remainder theorem (CRT) and applied it in FL. Yang et al. [35] further accelerated Paillier performance using field-programmable gate array (FPGA), enhancing efficiency from a hardware perspective. BatchCrypt [14] introduced a novel quantization encoding method, allowing a single Paillier ciphertext to carry more plaintext information and improving efficiency. However, this encoding method risks overflow in ciphertext addition, potentially compromising model accuracy. Later, Stripelis et al. [36] employed the lattice-based HE scheme to protect model updates, further enhancing security. Similarly, Xu et al. [37] used Cheon-Kim-Kim-Song (CKKS) [43] in cloud-edge FL but required that cloud and edge servers remain noncolluding. Han and Yan [38] proposed a fuzzy logic-based key selection algorithm designed to allocate keys of different sizes to clients with heterogeneous data and varying system processing capabilities. This approach improved the convergence speed of HE-based FL in heterogeneous scenarios. However, these schemes adopted single-key HE systems, which are vulnerable to privacy breaches in multidevice FL settings. Specifically, in single-server architectures, they are susceptible to internal device attacks and collusion between devices and the server. In multiserver architectures, they fail to defend against server–server collusion.

To address the privacy needs of multidevice FL, multikey HE has gradually been applied. Ma et al. [15] introduced xMK-CKKS, an improved MK-CKKS [44]. They used an aggregated public key for encryption and allowed decryption solely on aggregated ciphertexts, so that no party could decrypt individual model updates. ~~However~~, xMK-CKKS has a limitation in that it cannot tolerate offline devices, as decryption fails if a device disconnects. To address this issue, Du et al. [39] developed tMK-CKKS, which combines secret sharing with xMK-CKKS, allowing partial device disconnection and improving efficiency. Cai et al. [41] proposed SecFed, which employs an improved MK-BFV [44] to protect privacy and leverages a TEE to handle offline device issues. However, the frequent interactions with the TEE increase system overhead. Additionally, Li et al. [42] proposed a proxy reencryption scheme in which each device encrypts using its own public key, and the server converts ciphertexts under different public keys into ciphertexts under a single public key. However, the security of this scheme relies on the assumption that two servers do not collude. Yang et al. [40] designed a multiprivate-key secure aggregation algorithm based on the Diffie–Hellman key exchange. Additionally, they introduced a super-increasing sequence to compress gradient sizes, reducing both computational and communication costs. However, this scheme only supports homomorphic addition. In summary, while single-key HE falls short of multidevice privacy requirements in FL, multikey HE introduces higher computational and communication complexity and struggles to address device disconnection issues. Consequently, there is an urgent need for a single-key HE scheme capable of protecting multidevice privacy.

III. Preliminaries

In this section, we introduce the preliminary knowledge of FL as well as the cryptographic techniques required for our proposed framework.

A. Federated Learning

FL is a distributed machine learning approach that enables multiple devices to collaboratively train a global model without sharing raw data. Each remote device trains a local model on its dataset $\mathcal{D}_{i\in[1,n]}$ and sends the model updates (weights or gradients) to a central server. The server aggregates these updates to form the global model, which is then distributed back to devices for further training. The commonly used FedAvg algorithm [45] updates the global model by averaging the client model updates according to

$$w^{t+1} = \frac{1}{n}\sum_{i=1}^{n} w_i^{t+1} \tag{1}$$

where w^{t+1} is the global model after round $t+1$, w_i^t is the local model from device i, and n represents the total number of devices. Each device performs a local update by

$$w_i^{t+1} = w^t - \eta \nabla f_i\left(w^t, \mathcal{D}_i\right) \tag{2}$$

where η is the learning rate, $f(w^t, \mathcal{D}_i)$ is the local loss function. The goal in each round is to minimize the global loss function $F(w)$ by

$$\min F(w) = \frac{1}{n}\sum_{i=1}^{n} f_i(w) \tag{3}$$

$$f_i(w) = \frac{1}{|\mathcal{D}_i|}\sum_{(x,y)\ \in \mathcal{D}_i} \ell(w; x, y) \tag{4}$$

where $\ell(w; x, y)$ is the individual loss for data point (x, y), $|\mathcal{D}_i|$ is the number of samples in the $\mathcal{D}_i$.

Authorized licensed use limited to: Anhui University. Downloaded on March 24,2026 at 04:23:30 UTC from IEEE Xplore. Restrictions apply.

B. Fully Homomorphic Encryption

Fully HE (FHE) [46] is an essential technique for secure computation, allowing third parties to carry out operations, such as addition, subtraction, multiplication, and division, directly on encrypted data without accessing the underlying inputs or the computed results. Among various FHE schemes, CKKS [43] is particularly appealing for FL due to its approximate arithmetic mechanism. Traditional lattice-based FHE schemes (e.g., BFV and BGV) perform computations on integers or polynomials with exact arithmetic, requiring additional steps (such as fixed-point encoding) when dealing with floating-point data. These extra steps can introduce overhead and potentially reduce precision. In contrast, CKKS natively supports floating-point (approximate) arithmetic, making the encryption and computation process far more efficient for FL tasks. Furthermore, CKKS enables single instruction multiple data (SIMD) operations, wherein multiple data points (e.g., vectors of model updates) can be packed into a single ciphertext. This parallelism significantly reduces the computation and communication overhead within each aggregation step. Finally, CKKS's security relies on the ring LWE (RLWE) problem, offering a post-quantum resistance. Therefore, in this article, we utilize CKKS to secure model updates in FL.

Let $\mathcal{R} = \mathbb{Z}[X]/(X^N + 1)$, which $\mathcal{R}$ is the polynomial ring with integer coefficients. Set the ciphertext modulus q, key distribution χ and error distribution ψ on $\mathcal{R}_q$. Assume the model update from a single device is $g = (g_1, g_2, \ldots, g_{N/2}) \in \mathbb{C}^{N/2}$, where $\mathbb{C}$ represents the complex field. Sample $s, v \leftarrow \chi$, $a \leftarrow \mathcal{R}_q$, and $e, e_0, e_1 \leftarrow \psi$. Let integer $P = P(\lambda, q)$. Sample $a' \leftarrow \mathcal{R}_{P \cdot q}$, $e' \leftarrow \psi$. The CKKS scheme consists of seven algorithms: 1) KeyGen; 2) Encoding (Ecd); 3) Decoding (Dcd); 4) Encryption (Enc); 5) Decryption (Dec); 6) Addition (Add); and 7) Multiplication (Mult). The details are as follows.

1) *KeyGen*$(1^\lambda) \rightarrow (pk, sk, evk)$*:* Given the security parameter λ. The algorithm outputs a public key $pk \leftarrow (b, a) \in \mathcal{R}_q^2$, where $b \leftarrow -a \cdot s + e$, a private key $sk \leftarrow (1, s)$ and a evaluation key $evk \leftarrow (b', a')$, where $b' \leftarrow -a' \cdot s + e' + Ps^2$.
2) *Ecd*$(g) \rightarrow (g(x))$*:* Given a local model update vector $g \in \mathbb{C}^{N/2}$. The algorithm outputs an integer polynomial $g(x) \in \mathcal{R}$.
3) *Enc*$(g(x), pk) \rightarrow (c)$*:* For a plaintext $g(x)$, this algorithm encrypts it by using pk and outputs a ciphertext $c = (c_0, c_1) = (v \cdot b + g(x) + e_0, v \cdot a + e_1) \in \mathcal{R}_q^2$.
4) *Add*$(c, c') \rightarrow (c_{add})$*:* For two ciphertexts $c = (c_0, c_1)$ and $c' = (c_0', c_1')$, this algorithm produces a resulting ciphertext $c_{add} = c + c' = (c_0 + c_0', c_1 + c_1')$.
5) *Mult*$(c, c', evk) \rightarrow (c_{mult})$*:* Given ciphertexts c and c', along with the evaluation key evk, this algorithm generates a new ciphertext $c_{mult} = c \times c'$.
6) *Dec*$(c, sk) \rightarrow (g(x))$*:* For a ciphertext $c = (c_0, c_1)$, this algorithm decrypts it by using sk and outputs a plaintext $g(x) = c \cdot sk = c_0 + c_1 \cdot s$.
7) *Dcd*$(g(x)) \rightarrow (g)$*:* Given a plaintext $g(x) \in \mathcal{R}$. The algorithm outputs a complex vector $g \in \mathbb{C}^{N/2}$.

C. Elliptic Curve Cryptography

ECC [47] is a public-key cryptography method based on the algebraic structure of elliptic curves over finite fields. ECC provides strong security with smaller key sizes, making it computationally efficient and well-suited for resource-constrained environments, such as IoT devices. In this article, the ECC-based public key encryption algorithm is used to securely store the CKKS private key. We assume the base point of ECC is E, the order of the base point is $|E|$, the public key of ECC is Q and the private key of ECC is d. ECC operations are performed over a finite field modulo p, where p is a prime number. Given a plaintext m, the encryption and decryption processes of ECC are shown in

$$c = (c_0, c_1) = (r \cdot E, m + r \cdot Q) \tag{5}$$

$$m = c_1 - d \cdot c_0 = m + rQ - rE \cdot d = m + rQ - rQ \tag{6}$$

where r is the ephemeral key.

D. Secret Sharing

Shamir's linear secret sharing scheme (LSSS) [48] is utilized in our framework. LSSS splits secret y into n shares and ensures that any z or more shares can reconstruct secret y, while fewer than z shares reveal no information about it. The scheme is defined over a finite field $\mathbb{Z}_p$, where p is a large public prime number. It comprises two main algorithms: 1) *SS.share* and 2) *SS.recon*. In this article, we treat the ECC private key d as a secret and apply secret sharing to split d into multiple subsecrets, which are stored separately.

1) *SS.share*(y, z, n)*:* we randomly select $a_1, \ldots, a_{z-1} \in \mathbb{Z}_p$ and sets $a_0 = y$. Using these coefficients, we construct a polynomial of degree $z - 1$ as $f(x) = \sum_{j=0}^{j=z-1} a_j x^j$ $(mod\ p)$. For each participant, we compute $y_i = f(i)$ as their subsecret, where $i \in [1, n]$.
2) *SS.recon*(i, y_i)*:* We can reconstruct the secret y by collecting any z shares of the secret. $y = f(0) = \sum_{i=1}^{z} (\mu_i \cdot y_i) (\mathrm{mod}\, p)$, where $\mu_i = \prod_{\forall j \in [1,z], j \neq i} (-j/i - j)$.

IV. Models and Goals

In this section, we first provide an overview of the FL system model and the corresponding threat model adopted in this article, followed by a definition of our design objectives. The key notations used in this article and their definitions are summarized in Table II.

A. System Model

As shown in Fig. 1, the system model consists of four entities: 1) a trusted authority (TA); 2) a cloud node (CN); 3) ENs; and 4) end devices. The detailed information about these entities is described as follows. Furthermore, our solution is designed for cloud-edge IoT FL but can be easily adapted to the traditional single-server model. The cloud-edge architecture, however, aligns more closely with the mainstream requirements of IoT environments.

TA: The TA is responsible for generating and distributing the keys of CKKS and ECC. Additionally, the TA handles

Authorized licensed use limited to: Anhui University. Downloaded on March 24,2026 at 04:23:30 UTC from IEEE Xplore. Restrictions apply.

TABLE II
NOTATIONS AND CORRESPONDING DEFINITIONS

Notation	Definition
n	The number the of total end devices.
S	The number of participants at each aggregation round.
M	The number of the edge nodes.
z	The threshold of secret sharing.
$\mathcal{D}_i$	The local dataset of device i.
η	The local learning rate.
T_g	The number of aggregation rounds.
T_l	The number of local training epochs.
$\lvert EN_j \rvert$	The number of model updates collected by edge node j.
$\lvert \mathcal{D}_i \rvert$	The number of samples in the local dataset $\mathcal{D}_i$.
w_i^t	The local model of device i during the round t.
w^t	The global model at round t.
g_i^t	The local model update of devices i at round t.
$\overline{g}_j^t$	The partial aggregation result of edge node j at round t.
g^t	The global model update at round t.
pk_x/sk_y	The public key, private key of CKKS held by end devices.
$pk_y/[\![sk_x]\!]$	The public key, private key of CKKS held by CN.
Q	The public key of ECC.
d	The private key of ECC.
$[\![x]\!]$	The ciphertext of x.
ξ_i^t	The hash value of the local model update g_i^t.
$F(w)$	The global loss function.
$f_i(w)$	The local loss function of the device i.

the division of the ECC private key using secret sharing and distributes the resulting subsecrets to all ENs.

CN: The CN is a powerful server equipped with a TEE. Its primary responsibilities include collecting the decryption shares and partially aggregated results uploaded by the ENs and performing the global aggregation. Furthermore, all decryption operations are conducted within the TEE. Once the global aggregation is complete, the global model is distributed to the ENs.

ENs: ENs are edge servers commonly found in IoT environments. These servers are responsible for collecting model updates uploaded by remote devices and performing partial aggregation. Additionally, ENs calculate decryption shares based on the received subsecrets. When uploading the partially aggregated results to the CN, the ENs simultaneously upload the corresponding decryption shares.

End Devices: Remote devices are the data owners who are unwilling to upload their raw data due to privacy concerns. These devices train their models locally and then encrypt the model updates before uploading them to the ENs. Subsequently, they receive the aggregated global model from the ENs and benefit from it.

B. Threat Model

In this article, we primarily consider an honest-but-curious threat model. Malicious attacks in FL, such as poisoning attacks, are beyond the scope of this work. The attack capabilities of each entity in the system are summarized as follows.

Honest-But-Curious Servers: We assume that both ENs and the CN operate under an honest-but-curious threat model. While they honestly execute the algorithms and protocols as specified, they may attempt to infer additional information from the received model updates, thereby compromising the privacy of the devices. Furthermore, we consider the possibility of collusion between ENs and the CN, meaning that they might share information with each other to further compromise the privacy of devices.

Honest-But-Curious Devices: We assume that remote devices also follow an honest-but-curious threat model. They train their models and submit model updates honestly, but may collude with ENs, sharing additional information to compromise the privacy of other devices. Additionally, collusion among devices themselves is also considered a possibility.

TA: The TA is an independent entity, that is, trusted by both devices and servers. In fact, the assumption of such a TA is common in privacy-preserving FL [19], [39], [49].

C. Design Goals

In this article, our design aims to ensure that no semi-honest entity, even in cases of collusion, can compromise the privacy of devices. Moreover, we focus on achieving high efficiency in our solution. Our framework also supports the dynamic participation of devices, allowing them to exit the system at any time. Importantly, we aim to enhance privacy without sacrificing model accuracy. Therefore, based on the aforementioned system model and threat model, our design goals can be summarized into the following three key points.

Privacy Preserving: Directly uploading local model updates can still lead to privacy leakage. While single-key HE can address this issue, it is ineffective against collusion attacks. Therefore, the privacy protection requirements in this article are more stringent. Our goal is to defend against internal attacks among devices, collusion attacks between devices and ENs, as well as collusion attacks between ENs and the CN.

Efficiency: We use single-key CKKS to protect model updates, which is significantly more efficient than multikey HE. Although we introduced an innovative key management mechanism and TEE to address the limitations of single-key CKKS, the additional overhead introduced by TEE for secure operations is almost negligible.

Robustness: Multikey HE enhances security in multidevice environments but does not tolerate device offline scenarios, which is a critical limitation in IoT settings. In contrast, our solution allows devices to go offline at any time, making it more suitable for IoT environments. Moreover, multikey HE schemes can lead to model accuracy loss, which is a problem we aim to avoid.

V. OUR SCHEME

In this section, we provide a detailed explanation of our proposed efficient FL framework for privacy-preserving.

A. Scheme Overview

We structure our scheme into three algorithmic phases: 1) *system initialization*; 2) *local training*; and 3) *secure aggregation*. During the *system initialization* phase, the TA generates and distributes the necessary cryptographic keys. Additionally, the CN initializes and distributes the global model. This process corresponds to steps (1)–(4) in Fig. 1. System initialization is executed only once throughout the

Authorized licensed use limited to: Anhui University. Downloaded on March 24,2026 at 04:23:30 UTC from IEEE Xplore. Restrictions apply.

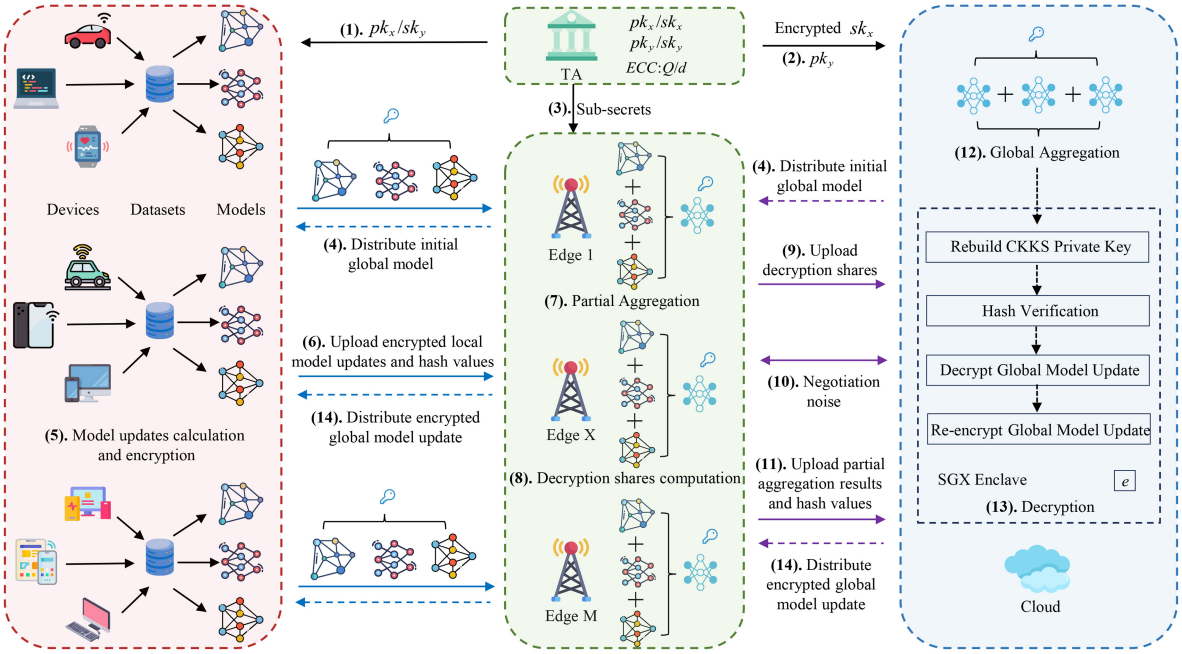


Fig. 1. System model.

entire FL process. Once *system initialization* is complete, remote devices proceed to the *local training* phase. In this phase, each device computes and then encrypts its model update and uploads ciphertexts to its corresponding EN. This process is illustrated in steps (5) and (6) of Fig. 1. Next, the system enters the *secure aggregation* phase. ENs perform partial aggregation, negotiate noise with the TEE, and compute decryption shares for the CKKS private key. These results are then uploaded to the CN. The CN performs global aggregation and, within the TEE, verifies, decrypts, and reencrypts the global model update. The secure aggregation process is represented by steps (7)–(13) in Fig. 1. Finally, the CN distributes the reencrypted global model update, as shown in step (14) of Fig. 1. Upon receiving the encrypted global model update, each remote device decrypts it using sk_y and proceeds to the next training round until convergence. The overall workflow of our scheme is presented in Algorithm 1, with detailed key algorithm protocols elaborated in the following sections.

Algorithm 1 FL-Based Single-Key CKKS Work Flow

Input: Security parameter λ, total number of rounds T_g.
Output: The FL task completed.

System Initialization(λ)
for t in $[1, T_g]$ **do**
  CN randomly selects S devices to participate in the current aggregation round;
  for $device_i \in devices$, $i \in [1, S]$ **do**
    $[\![g_i^t]\!], \xi_i^t \leftarrow$ ***Local Training***$(w^{t-1}, \mathcal{D}_i, \eta, T_l)$;
  end for
  $[\![g^t]\!] \leftarrow$ ***Secure Aggregation***$([\![g_i^t]\!], \xi_i^t)$, $i \in [1, S]$;
  $g^t \leftarrow \mathbf{Dcd}(\mathbf{Dec}([\![g^t]\!], sk_y))$;
  $w^t = w^{t-1} + g^t$;
end for
return The FL task completed.

B. System Initialization

In the system initialization phase, the main tasks include KeyGen and distribution. Additionally, secret shares are calculated and distributed. The initial global model is also generated and distributed during this phase. The detailed process is outlined in Algorithm 2.

1) ~~KeyGen and Distribution:~~ In this article, we utilize the CKKS and ECC cryptographic schemes. The TA generates public and private keys for CKKS and ECC based on the security parameters λ. Subsequently, the CKKS private key is encrypted using the ECC public key, as shown in line 6 of Algorithm 2. Finally, the TA distributes the encrypted CKKS private key to the CN.

2) ~~Subsecrets Computation and Distribution:~~ The TA splits the ECC private key into M subsecrets using a secret sharing scheme, where any z shares can reconstruct the private key. After calculating the secret shares, the TA distributes them to the respective ENs.

3) ~~Initial Global Model Distribution:~~ Before the start of FL, the CN generates an initial global model w^0 and distributes it to each remote device via ENs.

C. Local Training

After obtaining global model w, the remote devices train the model on their local datasets $\mathcal{D}$. They then compute the model updates, encrypt the updates using the CKKS public key pk_x, and upload them to their respective ENs. The detailed process of this phase is outlined in Algorithm 3.

~~1) Local Update:~~ The device i trains the model using SGD based on its own dataset $\mathcal{D}_i$, where $i \in [1, n]$. The specific training process is detailed in line 3 of Algorithm 3.

Authorized licensed use limited to: Anhui University. Downloaded on March 24,2026 at 04:23:30 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2 System Initialization

Input: Security parameter λ.

Output: CKKS keys (pk_x, sk_x), (pk_y, sk_y), encrypted CKKS private key $[\![sk_x]\!]$, ECC public key Q, ECC private key d, initial global model w^0.

1: /*Keys Generation and Distribution*/
2: *TA* generates CKKS keys $(pk_x, sk_x) \leftarrow$ **KeyGen**(1^λ);
3: *TA* generates CKKS keys $(pk_y, sk_y) \leftarrow$ **KeyGen**(1^λ);
4: *TA* generates ECC keys (d, Q);
5: *TA* encrypts sk using ECC public key Q;
6: $[\![sk_x]\!] \leftarrow (c_0, c_1) = (r \cdot E, sk_x + r \cdot Q)$;ECC公钥加密后
7: *TA* broadcasts the pk_x and pk_y;
8: TA distributes the $[\![sk_x]\!]$ to *CN*, sk_y to *devices*;ckks私钥密文分量
9: /*Subsecrets Computation and Distribution*/
10: $(i, y_i) \leftarrow$**SS.share**(d, z, M);ECC私钥秘密共享给边缘节点
11: TA distributes $(i,\ y_i)$ to *EN* i, for $i \in [1, M]$;
12: /*Initial Global Model Distribution*/
13: *CN* generates initial global model w^0;
14: **for** j in $[1, M]$ **do**
15: $EN_j \in ENs$ downloads w^0 from *CN*;
16: **end for**
17: **for** i in $[1, n]$ **do**
18: *Device* i downloads w^0 from *ENs*;
19: **end for**
20: **return** The *CN* obtains encrypted CKKS private key $[\![sk_x]\!]$. The *EN* i obtains subsecrets (i, y_i), for $i \in [1, M]$. *devices* obtain the initial model w^0 and CKKS private key sk_y.

Algorithm 3 Local Training

Input: Global model w, local dataset $\mathcal{D}$, local learning rate η, local training epochs T_l.

Output: Encrypted model update $[\![g]\!]$, hash value ξ of $[\![g]\!]$.

1: /*Local Update*/
2: $w^0 \leftarrow w$
3: **for** b in $[1, T_l]$ **do**
4: $w^b \leftarrow w^{b-1} - \eta \nabla f(w^{b-1}, \mathcal{D})$;
5: **end for**
6: /*Model Update computing*/
7: $g \leftarrow w^{T_l} - w^0$, where $g \in \mathbb{R}^{1\times d}$;
8: $[\![g]\!] \leftarrow$ **Enc**(**Ecd**$(g), pk_x)$;
9: $\xi =$ **HASH**$([\![g]\!])$;
10: **return** $[\![g]\!]$, ξ.

2) Model Update Computation: Remote device i calculates its local model update according to the formula in line 6 of Algorithm 3. Then, the local model update is encrypted using CKKS public key pk_x.

Additionally, in this article, we require the devices to compute a hash value for encrypted model updates and encrypt the number of samples in their local datasets $[\![|\mathcal{D}_i|]\!]$. These values are then uploaded alongside the encrypted model updates. **HASH**$(\cdot)$ denotes the hash function. The detailed definitions of **KeyGen**$(\cdot)$, **Ecd**$(\cdot)$, **Enc**$(\cdot)$, **Dcd**$(\cdot)$, **Dec**$(\cdot)$, and **SS.share**$(\cdot)$ can be found in Section III.

Algorithm 4 Secure Aggregation

Input: Local model updates $([\![g_1^t]\!], [\![g_2^t]\!], \ldots, [\![g_n^t]\!])$, hash values $(\xi_1^t, \xi_2^t, \cdots \xi_n^t)$.

Output: The global model update $[\![g^t]\!]$ at round t.

1: /*Partial Aggregation*/
2: **for** $EN_j \in ENs$ **do**
3: $[\![\overline{g}_j^t]\!] \leftarrow \frac{\sum_{i=1}^{|EN_j|} [\![|\mathcal{D}_i|]\!] \cdot [\![g_i^t]\!]}{\sum_{i=1}^{|EN_j|} [\![|\mathcal{D}_i|]\!]}$, for $j \in [1, M]$;
4: $\xi_j^t \leftarrow$ **HASH**$([\![\overline{g}_j^t]\!])$, for $j \in [1, M]$;
5: **end for**
6: Sent $([\![\overline{g}_1^t]\!], [\![\overline{g}_2^t]\!], \ldots, [\![\overline{g}_M^t]\!])$ to *CN*;
7: Sent $(\xi_1^t, \xi_2^t, \ldots, \xi_{n+M}^t)$ to CN;
8: /*Decryption Share Computation*/
9: **for** $EN_j \in ENs$ **do**
10: $DS_j = c_0 \cdot (\mu_j \cdot y_j) + e_j$, for $j \in [1, M]$;
11: **end for**
12: Sent $(DS_1, DS_2, \ldots, DS_M)$ to *CN*;
13: /*Global Aggregation*/
14: $[\![g^t]\!] = \frac{1}{M} \sum_{j=1}^{M} [\![\overline{g}_j^t]\!]$;
15: /*Global Model Decryption*/
16: **IN TEE:**
17: $sk_x \leftarrow c_1 - \sum_{i=1}^{z} DS_i - e$;
18: $\xi' =$ **HASH**$([\![g^t]\!])$
19: **for** $i \in [1, n+M]$ **do**
20: **if** $\xi_i == \xi'$ **then**
21: **return** TEE refuses to return any value;
22: **end if**
23: **end for**
24: $g^t \leftarrow$ **Dcd**(**Dec**$([\![g^t]\!], sk_x)$);
25: $[\![g^t]\!] \leftarrow$ **Enc**(**Ecd**$(g^t), pk_y)$;
26: **return** $[\![g^t]\!]$.

D. Secure Aggregation

In the secure aggregation phase, we provide an in-depth explanation of how our proposed framework ensures privacy. Within the cloud-edge FL architecture, ENs handle partial aggregation tasks, while the CN is responsible for global aggregation. Building upon this workflow, our framework incorporates several enhancements to strengthen privacy protection.

First, on the ENs, their role extends beyond partial aggregation to include the computation of decryption shares for the encrypted CKKS private key. Second, for the CN, a TEE is introduced to ensure security. Both the decryption of the CKKS private key and the decryption of the global model are performed entirely within the TEE on the CN. The process of the secure aggregation phase is summarized in Algorithm 4.

1) Partial Aggregation: For EN j, upon receiving the encrypted model updates from the devices at round t, it performs partial aggregation on the ciphertexts. The aggregation process is described by

$$[\![\overline{g}_j^t]\!] = \frac{\sum_{i=1}^{|EN_j|} [\![|\mathcal{D}_i|]\!] \cdot [\![g_i^t]\!]}{\sum_{i=1}^{|EN_j|} [\![|\mathcal{D}_i|]\!]} \tag{7}$$

where $|EN_j|$ represents the number of model updates collected by EN $j \in [1, M]$. $|\mathcal{D}_i|$ denotes the number of samples in the local dataset, where $i \in [1, n]$.

Authorized licensed use limited to: Anhui University. Downloaded on March 24,2026 at 04:23:30 UTC from IEEE Xplore. Restrictions apply.

2) Decryption Share Computation: It is important to note that the CKKS private key sk_x is stored in encrypted form on the CN using ECC, while the subsecrets of the ECC private key are distributed among the ENs. This setup ensures that no single entity can obtain the complete CKKS private key independently. However, after the global aggregation, the complete CKKS private key is required to decrypt the aggregated global model. Therefore, each EN needs to assist in reconstructing the CKKS private key. Each EN have (i, y_i) for $i \in [1, M]$. The ciphertext of sk_x is $[\![sk_x]\!] = (c_0 = r \cdot E, c_1 = sk_x + r \cdot Q)$. The decryption structure is represented $c_1 - c_0 \cdot d$, where d is the private key of ECC.

A straightforward approach is for at least z ENs to execute **SS.recon**$(\cdot)$ to recover the ECC private key d, which is then used to decrypt the encrypted CKKS private key. The detailed process is shown in (8) and (9). This represents the standard approach for ECC decryption based on secret sharing

$$d = \sum_{i=1}^{z} \mu_i \cdot y_i, \quad \mu_i = \prod_{\forall j \in [1,z], j \neq i} \left(\frac{-j}{i-j} \right). \tag{8}$$

$$sk_x = c_1 - c_0 \cdot d. \tag{9}$$

However, this method explicitly reconstructs the ECC private key, which increases the risk of private key exposure. To resolve this issue, we design a novel ECC decryption algorithm based on secret sharing, where the ECC private key is never explicitly reconstructed. This is a key feature of our approach. In our approach, each EN computes its decryption share as shown in

$$DS_i = c_0 \cdot (\mu_i \cdot y_i) + e_i \tag{10}$$

where $\mu_i = \prod_{\forall j \in [1,z], j \neq i} (-j/i-j)$, e_i represents a randomly sampled noise. Next, we provide a detailed explanation of the selection range for e_i. Let E be the base point of ECC, and let $|E|$ denote the order of the base point. Specifically, e_i is defined as $e_i = \alpha \cdot E$. To ensure the unpredictability of noise and the security of the system, α should be uniformly randomly sampled from the set $\{1, 2, \ldots, |E| - 1\}$

$$sk_x = c_1 - \sum_{i=1}^{z} DS_i - e. \tag{11}$$

~~Subsequently~~, each EN uploads the edge aggregation result along with its decryption share DS_i to the CN. The private key sk_x is fully reconstructed by summing any z decryption shares, as shown in (11). The proof of decryption correctness is provided in

这里有错误
应该是+e

$$\begin{aligned} sk_x &= c_1 - \sum_{i=1}^{z} DS_i - e \\ &= c_1 - \sum_{i=1}^{z} c_0 \cdot (\mu_i \cdot y_i) + e_i - e \\ &= c_1 - c_0 \cdot \sum_{i=1}^{z} \mu_i \cdot y_i + \sum_{i=1}^{z} e_i - e \\ &= c_1 - c_0 \cdot d + e - e \\ &= sk_x. \end{aligned} \tag{12}$$

3) Global Aggregation: Upon receiving the partial aggregation results uploaded by the ENs at round t, the CN performs global aggregation on the ciphertexts by

$$[\![g^t]\!] = \frac{1}{M} \sum_{j=1}^{M} [\![\bar{g}_j^t]\!]. \tag{13}$$

4) Global Model Decryption: In the regular memory of the CN, at least z decryption shares uploaded by the ENs have been received. Additionally, the aggregated global model update is stored in ciphertext form in the CN's regular memory. To decrypt the aggregated global model, the complete CKKS private key sk_x must be reconstructed. Although the private key is reconstructed only in the final stage, it still poses potential privacy risks. Ideally, the CN should be able to use the private key without gaining access to it, achieving the property of usability without visibility. This challenge is perfectly addressed by TEE. To ensure this, we delegate this stage of the task to the TEE for execution. First, at least z secret shares are transferred into the TEE to reconstruct the CKKS private key sk_x. The specific process for CKKS private key reconstruction is detailed in (12).

Note that $\sum_{i=1}^{z} e_i = e$. To enable the addition and removal of such noise, the TEE generates a randomly sampled noise value e_i for each EN, and these noise values are exchanged through a secure protocol. ~~Specifically~~, each EN can choose a public-key cryptography scheme based on its specific requirements, generate a public-private key pair, and broadcast the public key. The TEE assigns a randomly sampled noise value to each EN and encrypts it using the EN's broadcasted public key before distribution. Upon receiving the encrypted random noise, each EN decrypts it using its private key.

The CKKS private key has been successfully reconstructed within the TEE. Now, we need to load the aggregated global model into the TEE and decrypt it using the reconstructed CKKS private key. It is crucial to note that decryption cannot be performed without proper verification. If decryption is executed without validation, the CN could decrypt partial aggregation results, and in the case of collusion, it could still decrypt individual model updates.

~~To address this issue~~, we utilize the hash values of the model updates uploaded by the devices for verification. ~~Specifically~~, each device uploads a hash value computed from its model update. ~~Additionally~~, CNs also compute the hash values of the partially aggregated results. These hash values form a hash matrix $(\xi_1, \xi_2, \ldots, \xi_{n+M})$, which is synchronized to the CN. When the CN requests decryption from the TEE, the TEE first computes the hash value of the encrypted model update provided by the CN. It then compares the computed hash with the hash matrix. If a matching hash value is found in the hash matrix, it indicates that the model update is not the aggregated global model update. ~~In this case~~, the TEE aborts the process and refuses to return any value. ~~Conversely~~, if the hash value of the input model update does not exist in the hash matrix, the TEE decrypts the global model update using the reconstructed CKKS private key sk_x, reencrypts the model update with the CKKS public key pk_y, and returns the result to regular memory. ~~Subsequently~~, the CN distributes

Authorized licensed use limited to: Anhui University. Downloaded on March 24,2026 at 04:23:30 UTC from IEEE Xplore. Restrictions apply.

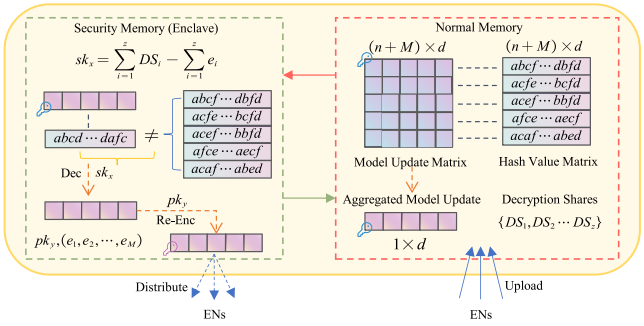


Fig. 2. Secure aggregation process on CN.

the encrypted global model update to all devices. devices can then decrypt the global model using their private key sk_y and continue training with it.

This mechanism ensures that the TEE decrypts only the aggregated global model, not individual model updates, thereby maintaining privacy even in the presence of collusion. The detailed process of secure aggregation on CN is illustrated in Fig. 2.

VI. Security Analysis

In this section, we conduct a comprehensive analysis of the security of our proposed framework. The security of our approach is defined under the assumption that the TEE is fully trusted.

Theorem 1 (Security Against Honest-But-Curious Devices): In our scheme, honest-but-curious devices cannot infer any personal information by eavesdropping on the shared data of other devices.

Proof: For an honest-but-curious device i, it holds the CKKS public key pk_x and CKKS private key sk_y. The model updates uploaded by the devices are encrypted using pk_x, and sk_y cannot be used to decrypt them. ~~Therefore,~~ an honest-but-curious device cannot infer the privacy of other devices.

It is important to note that in the tth aggregation round, all devices eventually obtain the same global model update g^t. If $n-1$ devices collude, they could attempt to infer the model update submitted by the remaining device from the global model update. ~~In our scheme~~, during partial aggregation, the encrypted dataset sample sizes $|\mathcal{D}_i|$ submitted by each device are used to perform a weighted average aggregation, rather than simple averaging or additive aggregation. As long as at least one device does not collude, the exact weighting coefficients cannot be recovered. ~~Therefore,~~ our scheme can tolerate collusion among up to $n-1$ devices. ■

Theorem 2 (Security Against Honest-But-Curious ENs): In our scheme, honest-but-curious ENs cannot infer any personal information from the shared data provided by the devices.

Proof: For an honest-but-curious EN j, it possesses only a secret share (j, y_j) of the ECC private key d and has no knowledge of any additional information. Therefore, it cannot independently decrypt the model updates uploaded by the devices.

A possible collusion scenario involves more than $z-1$ EN working together. However, even in this case, they can only reconstruct the ECC private key d but still cannot decrypt the model updates from the devices. Consequently, an honest-but-curious EN cannot infer the privacy of the devices. ■

Theorem 3 (Security Against Honest-But-Curious CN): In our scheme, an honest-but-curious CN cannot infer any personal information of devices from the shared data provided by the ENs.

Proof: The CN is divided into regular memory and secure memory. Secure memory is fully trusted and does not pose any privacy risks. In the tth aggregation round, the CN holds at least z decryption shares $(DS_1, DS_2, \ldots, DS_z)$ and partially aggregated results $([\![\overline{g}_1^t]\!], [\![\overline{g}_2^t]\!], \cdots, [\![\overline{g}_M^t]\!])$ in its regular memory. The CN may attempt to reconstruct the CKKS private key sk_x by summing the z decryption shares. However, these decryption shares are embedded with noise $(e_1, e_2, \ldots, e_z)$, which can only be removed within the secure memory. As a result, the CN cannot independently recover sk_x and does not have decryption authority.

A possible attempt might involve the CN sending the partially aggregated result $[\![\overline{g}_j^t]\!]$ to the TEE for decryption. However, the hash verification mechanism we propose ensures that such unauthorized decryption requests fail. Therefore, an honest-but-curious CN cannot infer the privacy of the devices. ■

Theorem 4 (Security Against Channel Eavesdroppers): In our scheme, eavesdroppers monitoring the communication channels cannot infer any information about the user datasets or model updates from the intercepted data.

Proof: The model updates submitted by the devices are encrypted using CKKS, which provides quantum-resistant security. Therefore, eavesdroppers cannot decrypt the ciphertext without the private key. Although the hash values of the model updates are transmitted without encryption, eavesdroppers gaining access to these hashes do not compromise the devices' privacy.

Additionally, while the decryption shares uploaded by the ENs are not encrypted, they are processed with added noise. Even if an eavesdropper intercepts all the uploaded decryption shares, it is impossible to correctly reconstruct the CKKS private key sk_x. In summary, under our scheme, channel eavesdroppers cannot compromise the privacy of the devices. ■

Theorem 5 (Security Against Collusion Between Devices and ENs): In our scheme, collusion between devices and ENs does not result in the leakage of sensitive device data.

Proof: As previously mentioned, devices hold the CKKS public key pk_x and private key sk_y, while ENs hold the subsecrets (i, y_i). In the worst-case scenario, more than $z-1$ ENs could potentially reconstruct the ECC private key d. However, as long as the CN is not compromised, collusion between devices and ENs cannot reconstruct the CKKS private key sk_x. Therefore, even if devices and ENs collude, they cannot decrypt the model updates of any device. ■

Theorem 6 (Security Against Collusion Between ENs and CN): In our scheme, up to $M-1$ EN can collude with the CN without compromising the sensitive data of the devices.

Proof: Before the CKKS private key sk_x is reconstructed, each EN uploads noisy e_i decryption shares DS_i according to the secure aggregation protocol. The noise will be removed

Authorized licensed use limited to: Anhui University. Downloaded on March 24,2026 at 04:23:30 UTC from IEEE Xplore. Restrictions apply.

TABLE III
CRYPTOGRAPHIC PARAMETERS

Scheme	Security Level	Key Size
BatchCrypt	128 bits	3072 bits
xMK-CKKS	128 bits	$N = 8192$ ($logq \leq 218$ bits)
SecFed(EMK-BFV)	128 bits	$N = 4096$ ($logq \leq 109$ bits)
Ours(CKKS)	128 bits	$N = 8192$ ($logq \leq 218$ bits)

TABLE IV
DATASETS AND CORRESPONDING MODELS

Dataset	Sample Number	Model	Parameter Number
MNIST	70000	2D-CNN	21840
FashionMNIST	70000	2D-CNN	44426
CIFAR10	60000	ResNet18	11191262
UCI-HAR	7352	1D-CNN	284904

within the TEE, reconstructing the complete sk_x. Collusion between the CN and ENs cannot extract the CKKS private key from the TEE, as the TEE is fully trusted.

A potential risk is that if more than $z-1$ ENs collude with the CN, they could reconstruct the ECC private key d and then decrypt the encrypted CKKS private key $[\![sk_x]\!]$ stored on the CN. As long as the number of colluding ENs and the CN does not reach the threshold z of the secret sharing scheme, the CN cannot reconstruct the CKKS private key outside the TEE.

Therefore, our scheme can tolerate up to $M-1$ colluding EN with the CN when the threshold of the secret sharing scheme is set to M. ■

VII. EXPERIMENT AND EVALUATION

In this section, we first introduce the basic setup of the experiments. Then, we compare our proposed scheme with advanced schemes, such as BatchCrypt [14], xMK-CKKS [15], and SecFed [41], evaluating our scheme from four aspects: 1) computational cost; 2) communication cost; 3) model accuracy; and 4) privacy preserving.

A. Experiment Setup

Hardware Setting: We use multiple laptops equipped with 12th Gen Intel Core i7-1260P CPUs and 8 GB of memory as the devices. The CN is hosted on an Alibaba Cloud SGX-enhanced compute instance, featuring an Intel Xeon Scalable Processor (Ice Lake) with a 3.5-GHz clock speed and 64 GB of memory. Multiple servers equipped with Intel Xeon W-2295 CPUs at 3.00 GHz and 32 GB of memory are used as ENs. Additionally, the devices are paired with NVIDIA GeForce MX150 GPUs to accelerate the model training process.

Software Setting: We implement the CKKS scheme in our solution using the TenSEAL [50] library, which is the Python version of the Microsoft SEAL [51] library. ~~Additionally~~, we use the Python-Paillier [52] library to implement BatchCrypt, the HEAAN library for xMK-CKKS, and the SEAL [51] library for SecFed. The specific parameters for these cryptographic methods are summarized in Table III. All experiments are conducted on an Ubuntu 20.04 operating system, with model training performed using the PyTorch framework.

TEE Setting: We deploy the TEE using Intel SGX. SGX provides a secure area in memory called an Enclave, which is inaccessible to other processes. ~~However~~, developing applications for SGX can be quite complex, and SGX does not natively support running Python programs. ~~To address these challenges~~, we use Occlum [53], which allows Python programs to run directly within SGX, thereby improving efficiency.

FL Setting: We evaluate our approach using four benchmark datasets: 1) MNIST [54]; 2) FashionMNIST [55]; 3) CIFAR10 [56]; and 4) UCI-HAR [57]. MNIST, FashionMNIST, and CIFAR10 are commonly used image classification datasets in the field of FL. In fact, 2-D image datasets are also commonly found in real-world IoT environments. For example, in smart agriculture, remote camera devices capture images of crops to detect and identify pests and diseases. The UCI-HAR dataset, which is collected from smartphone-embedded accelerometers and gyroscopes, is frequently employed in human activity recognition studies within IoT environments. The MNIST and FashionMNIST datasets are trained using a 2D-CNN model with parameter sizes of 21840 and 44426, respectively. The CIFAR-10 dataset is trained using ResNet18, which has 11191262 parameters. Unlike the above datasets, the UCI-HAR dataset consists of 1-D time-series data. Therefore, we use a 1D-CNN model with 284904 parameters for training. A summary of the datasets and their corresponding models is provided in Table IV. All training processes begin with an initial learning rate of 0.01, and there are 3 local training rounds per aggregation round.

B. Computational Cost

We compare our proposed scheme with BatchCrypt, xMK-CKKS, and SecFed, conducting a comprehensive evaluation of computational costs across various aspects, including KeyGen, device-side encryption, device-side decryption, ciphertext aggregation, and server-side decryption. The results are presented in Fig. 3.

1) KeyGen: Both BatchCrypt and our proposed scheme are based on single-key HE, where the computational cost of KeyGen does not increase with the number of devices. In contrast, xMK-CKKS and SecFed rely on multikey HE, where the computational cost of KeyGen grows with the number of devices. As shown in Fig. 3(a), our scheme achieves the lowest computational cost for KeyGen, improving efficiency by up to 98.4% compared to multikey HE-based schemes, demonstrating significant advantages.

2) Encryption and Decryption: Fig. 3(b) illustrates the computational costs of device-side encryption for different model sizes, while Fig. 3(c) shows the computational costs of device-side decryption for the same models. As indicated by the encryption parameters in Table III, a single ciphertext in our scheme can accommodate 4096 plaintext values. For a fair comparison, xMK-CKKS and SecFed are also configured to support 4096 plaintext values per ciphertext. BatchCrypt, which supports batch processing, encodes each parameter using 2 symbol bits and 2 padding bits, allowing a single ciphertext to hold 85 plaintext values.

Authorized licensed use limited to: Anhui University. Downloaded on March 24,2026 at 04:23:30 UTC from IEEE Xplore. Restrictions apply.

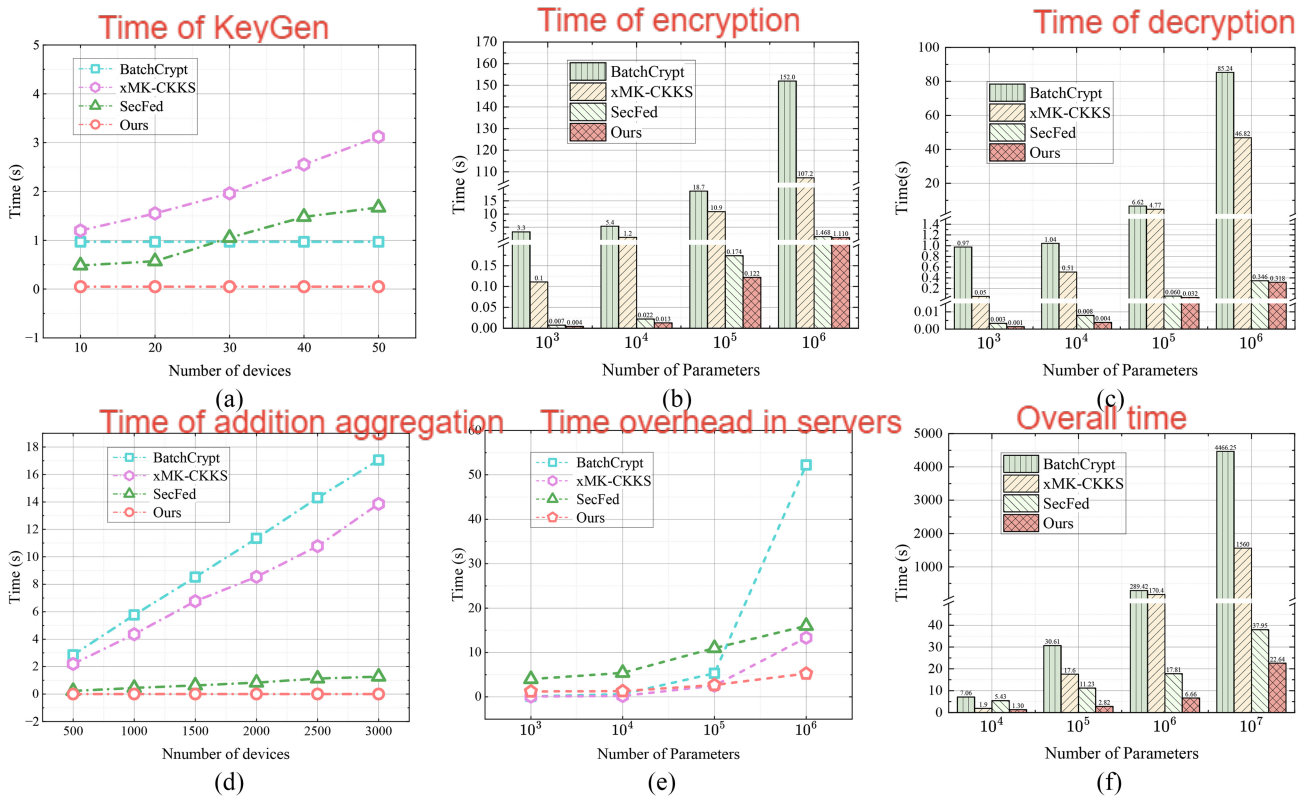


Fig. 3. Comparison of computational costs. (a) Time of KeyGen. (b) Time of encryption. (c) Time of decryption. (d) Time of addition aggregation. (e) Time overhead in servers. (f) Overall time.

As expected, BatchCrypt incurs the highest computational cost due to the limited capacity of its ciphertexts. Both xMK-CKKS and SecFed are based on multikey HE, but the operations at the device level still use single-key encryption and decryption. xMK-CKKS, however, involves joint key encryption and requires additional computation for decryption shares, resulting in significantly higher costs compared to SecFed.

Our scheme achieves the lowest computational costs due to CKKS's ability to directly operate on floating-point numbers, whereas SecFed requires encoding floating-point numbers into integers before performing operations. As shown in Fig. 3(b) and (c), our scheme improves encryption efficiency at the device side by a maximum of 99.27% and a minimum of 24.35%. For decryption, it achieves a maximum efficiency improvement of 99.6% and a minimum of 8.31%.

3) Addition Aggregation: We evaluated the computational costs of addition aggregation for different numbers of devices across various schemes, as shown in Fig. 3(d). In this test, the model size was set to 10^3. BatchCrypt remains the least efficient scheme, primarily due to its smaller batch processing capacity.

Although xMK-CKKS performs ciphertext addition under single-key encryption at its core, the use of an aggregated public key for multiparty aggregation significantly reduces its efficiency compared to our scheme. On average, our scheme achieves a 99.9% improvement in efficiency over xMK-CKKS.

SecFed, on the other hand, performs ciphertext addition under different encryption keys but leverages TEE to enhance its efficiency. While SecFed is more efficient than xMK-CKKS, it still falls short of our scheme. Compared to SecFed, our scheme delivers an average efficiency improvement of 99.6%.

In addition, we tested the computational cost of ciphertext multiplication in our proposed secure aggregation scheme, with the results shown in Table V. Since BatchCrypt and xMK-CKKS do not support ciphertext multiplication, and SecFed essentially performs plaintext multiplication, we did not include them in the comparison. The results indicate that ciphertext multiplication remains highly efficient when the model size and number of devices are relatively small.

4) Time Overhead in Servers: We evaluated the computational costs on the server side for each scheme, as shown in Fig. 3(e). As the model size increases, the advantages of our scheme become more pronounced. Note that in this test, the number of devices is fixed at 10.

BatchCrypt only performs ciphertext aggregation on the server, while xMK-CKKS performs ciphertext aggregation along with decryption share aggregation, giving them a certain efficiency advantage when the number of model parameters is small. SecFed, however, involves ciphertext aggregation on the server and requires frequent interactions with the TEE to refresh noise, introducing additional computational overhead.

In our scheme, CKKS private key reconstruction is performed within the TEE, resulting in a fixed time overhead, that is, independent of the model size. Nevertheless, as the number of model parameters increases, our scheme demonstrates significant advantages. When the model size reaches 10^6, our scheme improves efficiency by 60.8% compared to xMK-CKKS and by 67.3% compared to SecFed.

Authorized licensed use limited to: Anhui University. Downloaded on March 24,2026 at 04:23:30 UTC from IEEE Xplore. Restrictions apply.

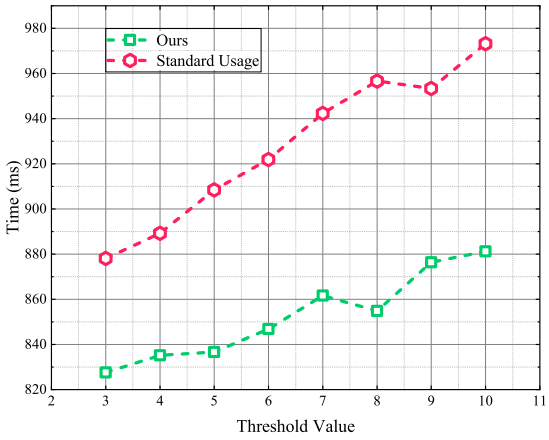


Fig. 4. Computational costs with varying threshold values.

TABLE V
TIME OF MULTIPLICATION

Model Size	Number of devices			
	10	20	30	40
10^3	5.43 ms	10.68 ms	15.91 ms	22 ms
10^4	20.40 ms	34.20 ms	49.93 ms	69.85 ms
10^5	145.97 ms	296.87 ms	509.62 ms	618.68 ms
10^6	1525 ms	3042 ms	6120 ms	21266 ms

The comparison of total time is presented in Fig. 3(f). We also evaluated the amortized computational cost per plaintext within a single ciphertext, as shown in Table VI. Due to BatchCrypt's smaller packing size, xMK-CKKS, SecFed, and our proposed scheme demonstrate a significant advantage in amortized computational cost. Among them, our scheme exhibits the greatest efficiency. Whether considering the time for individual operations or the total time, our scheme significantly improves computational efficiency. The results of the computational cost improvements for each round are summarized in Table VII.

The effect of the threshold z on computational costs was quantitatively analyzed to evaluate our system performance. As shown in Fig. 4, an increase in z leads to more ENs participating in decryption, thereby raising computational costs. However, experimental results indicate that the increase in computational overhead remains within a manageable range. When the threshold z increases from 3 to 10, the computational cost rises by only 6%. Additionally, we compared our proposed novel ECC decryption algorithm based on secret sharing with the traditional standard approach (Fig. 4). Our improvement eliminates one multiplication operation in the decryption process, enhancing decryption efficiency.

C. Communication Cost

We evaluated the communication costs by comparing the ciphertext memory size for each scheme. It is worth noting that in our scheme, devices also need to transmit the hash values of the model parameters, and the communication cost of these hash values is included in the results. Additionally, we measured the memory and time overhead within the TEE.

TABLE VI
AMORTIZED COMPUTATIONAL COST COMPARISON WITH OTHER SCHEMES

Scheme	Packing Size	Encryption Time (amortized) (ms)	Decryption Time (amortized) (ms)
BatchCrypt	85	73.1 (0.86)	25.5 (0.3)
xMK-CKKS	4096	450.5 (0.11)	204.8 (0.05)
SecFed	4096	28.7 (0.007)	12.3 (0.003)
Ours	4096	8.2 (0.002)	4.1 (0.001)

TABLE VII
COMPUTATIONAL SPEEDUPS COMPARED WITH OTHER SCHEMES

Model Size	vs BatchCrypt	vs xMK-CKKS	vs SecFed
10^4	5.44 ×	1.46 ×	4.18 ×
10^5	10.86 ×	6.24 ×	3.98 ×
10^6	43.44 ×	25.57 ×	2.67 ×
10^7	197.25 ×	68.91 ×	1.67 ×

TABLE VIII
COMMUNICATION REDUCTIONS COMPARED WITH OTHER SCHEMES

Model Size	vs BatchCrypt	vs xMK-CKKS	vs SecFed
10^5	15.98 ×	14.37 ×	1.66 ×
$5 \cdot 10^5$	15.99 ×	14.38 ×	1.64 ×
10^6	15.9 ×	14.49 ×	1.64 ×
$5 \cdot 10^6$	15.78 ×	14.41 ×	1.63 ×
10^7	15.89 ×	14.5 ×	1.64 ×

We also evaluated the impact of different threshold values z on communication costs. These results are presented in Fig. 5.

In Fig. 5(a), we present a comparison of ciphertext sizes for different model sizes. As shown, our scheme has an advantage in ciphertext size, improving communication efficiency. Both BatchCrypt and xMK-CKKS have larger ciphertexts, which results in no advantage in terms of communication efficiency. SecFed uses integer ciphertexts, while our scheme uses floating-point ciphertexts. Despite this, the ciphertexts in our scheme are still smaller than those of SecFed, further highlighting the efficiency of our approach. Compared to the other schemes, our approach achieves a minimum improvement of 38.9% and a maximum improvement of 93.7% in communication efficiency. As shown in Fig. 5(b) and (c), we also measured the total communication cost and the average communication cost for different numbers of participating devices (with a model size of 10^5). The results indicate that our scheme demonstrates a significant advantage in terms of average communication cost compared to other approaches. Moreover, as the number of participating devices increases, the superiority of our scheme in total communication cost becomes even more pronounced. The communication cost improvement results of our scheme compared to other schemes are summarized in Table VIII.

In addition, we evaluated the additional computational and communication costs introduced by TEE. Fig. 5(d) shows the consumption of secure memory. As the model size increases, the consumption of secure memory also rises. However, even when the model size reaches 10^7, the secure memory

Authorized licensed use limited to: Anhui University. Downloaded on March 24,2026 at 04:23:30 UTC from IEEE Xplore. Restrictions apply.

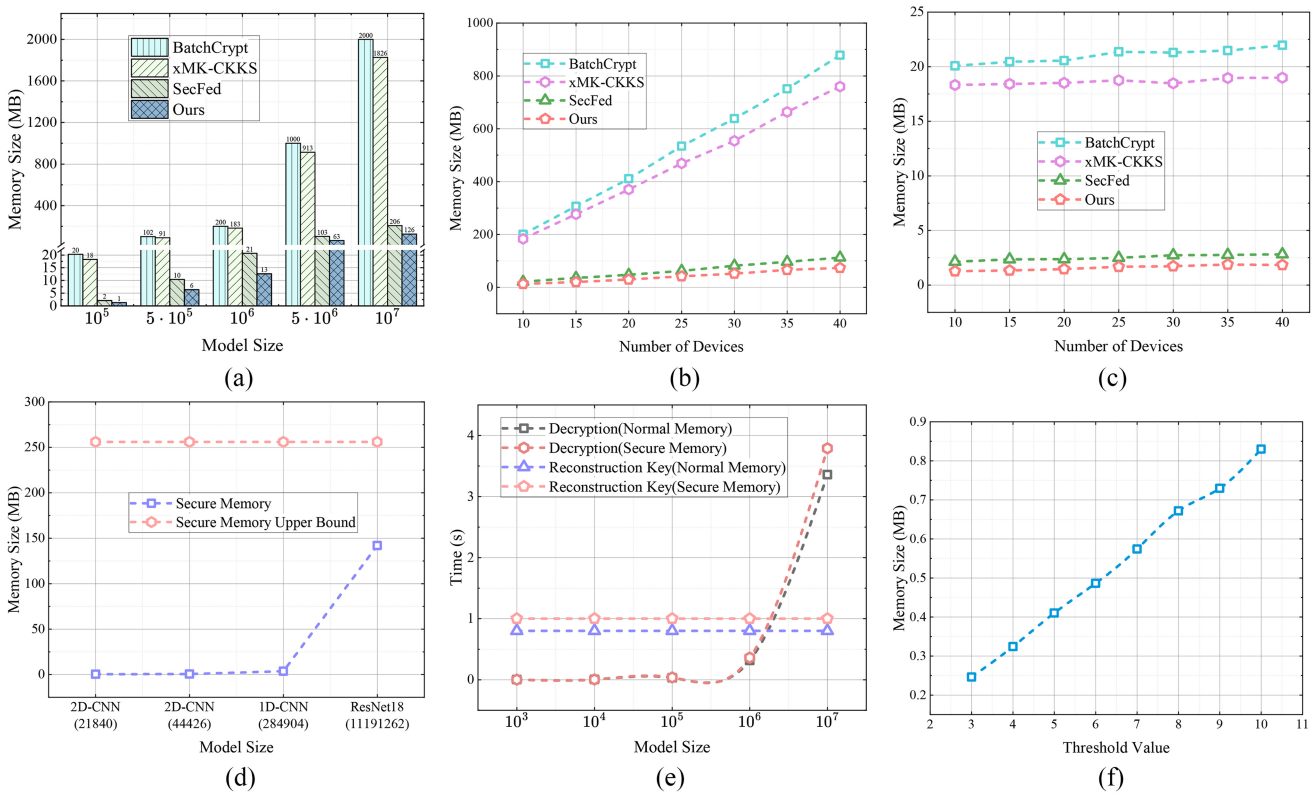


Fig. 5. Comparison of communication costs and TEE's costs. (a) Memory of ciphertexts. (b) Total communication costs with varying number of devices. (c) Average communication costs per device with varying number of devices. (d) Memory cost within TEE. (e) Time cost within TEE. (f) Communication costs with varying threshold values.

consumption is only 142.99 MB, well below the maximum secure memory limit (256 MB), which avoids costly paging operations and improves efficiency. Fig. 5(e) presents a comparison of the time costs for the same operations in secure memory versus regular memory. It is evident that the use of TEE does not incur high-time costs; the key reconstruction process only adds 0.2s, and the decryption process adds a maximum of only 0.43s.

To further access communication costs, we conducted a quantitative analysis of the impact of the threshold z on communication overhead. As shown in Fig. 5(f), as z increases, the number of decryption shares that must be uploaded also increases, leading to higher communication costs. Experimental results indicate that each decryption share incurs a communication cost of only 83–86 B, which accounts for just 8% of a single CKKS ciphertext (for a model size of 10^5). Therefore, the additional communication overhead introduced by increasing z remains acceptable compared to the system's primary communication cost, which is dominated by CKKS ciphertexts.

D. Model Accuracy

Next, we evaluated the model accuracy of each scheme, using FL without any defense mechanisms as the baseline. The results are shown in Fig. 6.

In Fig. 6(a), we present the model accuracy evaluation on the MNIST dataset. It can be observed that, except for BatchCrypt, the other schemes demonstrate negligible differences compared to the baseline. SecFed and xMK-CKKS incur an accuracy loss of 1%–2%, while our scheme exhibits no accuracy loss.

Fig. 6(b) shows the evaluation results on the FashionMNIST dataset. Again, BatchCrypt demonstrates the highest accuracy loss, while SecFed and xMK-CKKS show accuracy losses ranging between 1%–3%. In contrast, our scheme maintains accuracy identical to the baseline.

We also display the model accuracy results on the CIFAR10 dataset in Fig. 6(c). We find that BatchCrypt's accuracy loss increases further compared with the previous datasets, and both SecFed and xMK-CKKS also experience higher accuracy losses. However, our scheme still shows no difference compared to the baseline. The accuracy loss in BatchCrypt arises from its encoding and quantization process, which causes ciphertext aggregation to overflow, leading to reduced model accuracy. xMK-CKKS introduces significant noise for security purposes, impacting the decryption results and thereby reducing accuracy. SecFed, constrained by its integer-only encryption capability, loses precision during the encoding of floating-point numbers into integers, which affects model accuracy.

The evaluation results on the UCI-HAR dataset are presented in Fig. 6(d). BatchCrypt exhibits an accuracy loss of 5%–7%, while SecFed and xMK-CKKS show an accuracy loss of 1%–3%. In contrast, our scheme maintains model accuracy, which remains nearly identical to the baseline without defense.

Authorized licensed use limited to: Anhui University. Downloaded on March 24,2026 at 04:23:30 UTC from IEEE Xplore. Restrictions apply.

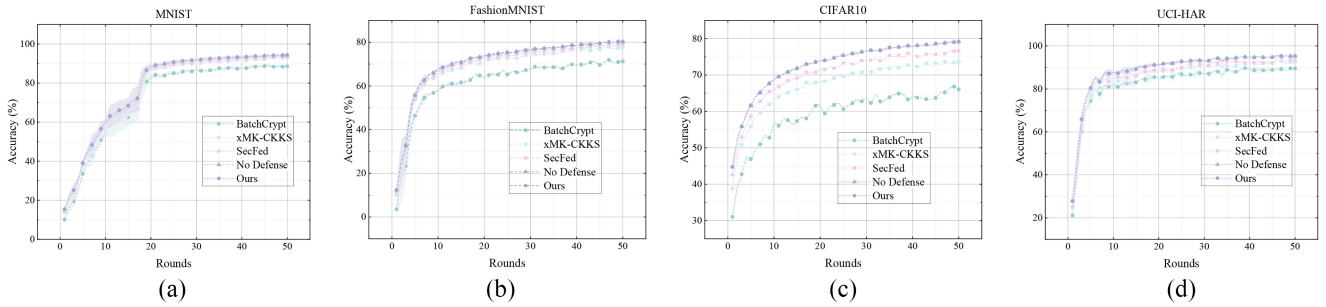


Fig. 6. Comparison of model accuracy on different datasets. (a) Model accuracy on MNIST. (b) Model accuracy on FashionMNIST. (c) Model accuracy on CIFAR10. (d) Model accuracy on UCI-HAR.

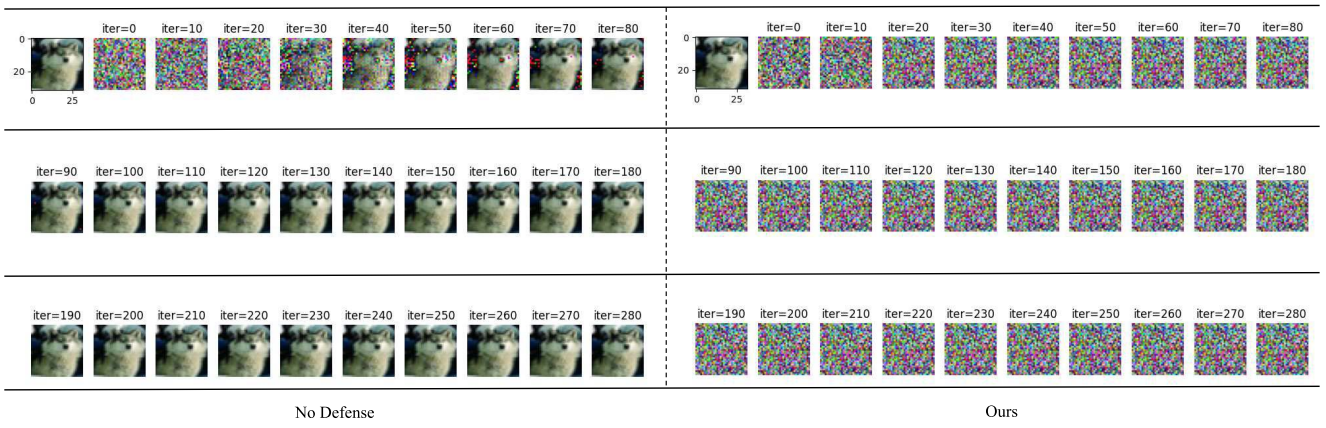


Fig. 7. Raw image exposure degree with iterations.

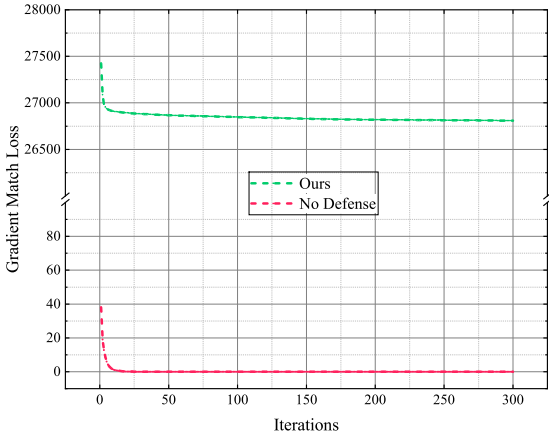


Fig. 8. Comparison of gradient match loss.

Our scheme, leveraging single-key CKKS, ensures that decryption results maintain full precision, resulting in virtually no accuracy loss compared to the baseline. These experimental results demonstrate that our scheme achieves its design objectives, offering security, efficiency, and robustness.

E. Privacy Preserving

To evaluate the degree of privacy protection, we replicated the HCGLA [58] attack, an improved version of the deep leakage from gradients (DLG) [7] attack. Under the HCGLA attack, the exposure levels of the original images in our scheme and the baseline without defense are shown in Fig. 7, while the corresponding gradient match loss is presented in Fig. 8.

Experimental results demonstrate that CKKS provides strong protection and significantly reduces the exposure of the original image. After 300 iterations of the HCGLA attack, the gradient match loss in our scheme remains on the order of 10^5, indicating that the original image remains completely unexposed. In contrast, the baseline without any defense exhibits a gradient match loss close to 0, resulting in a complete exposure of the original image. Encrypted ciphertexts under CKKS are relatively independent of their corresponding plaintexts, meaning that undecrypted model updates can be regarded as random values. However, single-key HE schemes struggle to safeguard the decryption private key, making them vulnerable to DLG attacks if the private key is compromised. Our scheme significantly enhances the security of single-key HE by ensuring that no untrusted party can obtain the complete decryption private key.

VIII. Conclusion and Future Work

In this article, we propose an efficient and privacy-enhanced FL framework tailored for cloud-edge-end architectures. Our key innovation is enhancing the security of single-key HE by securely storing the private key, while avoiding multikey HE

Authorized licensed use limited to: Anhui University. Downloaded on March 24,2026 at 04:23:30 UTC from IEEE Xplore. Restrictions apply.

to improve efficiency. Specifically, we use ECC to encrypt the CKKS private key and distribute the ECC private key across multiple ENs using secret sharing. This setup ensures that no single entity can independently reconstruct the full CKKS private key, maintaining security before the completion of global aggregation. After global aggregation, the CKKS private key must be reconstructed to decrypt the global model update. To achieve this, we designed an innovative ECC decryption algorithm based on secret sharing, where the ECC private key is never explicitly reconstructed, thereby improving decryption efficiency. To further ensure the security of the CKKS private key, this process is carried out within a TEE, achieving usability without visibility of the CKKS private key for the CN. The global model is also decrypted within the TEE, and to prevent misuse of the TEE's decryption capabilities, we introduced a hash verification mechanism, allowing only aggregated global model updates to be decrypted in the TEE. These measures ensure strong security during the global model decryption process. Our security analysis demonstrates that this framework is resilient against collusion attacks under the honest-but-curious threat model. Compared to schemes based on multikey HE, extensive experiments show that our approach significantly improves efficiency while achieving the same level of security. Additionally, the proposed framework supports offline device scenarios without affecting model accuracy, making it more suitable for IoT environments. For future work, we plan to explore malicious threat models, such as defending against poisoning attacks in FL.

REFERENCES

[1] M. Soori, B. Arezoo, and R. Dastres, "Internet of Things for smart factories in industry 4.0, a review," *Internet Things Cyber-Phys. Syst.*, vol. 3, pp. 192–204, Jan. 2023.

[2] Y. Fu, C. Li, F. R. Yu, T. H. Luan, and P. Zhao, "An incentive mechanism of incorporating supervision game for federated learning in autonomous driving," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 12, pp. 14800–14812, Dec. 2023.

[3] L. Zhang, J. Xu, P. Vijayakumar, P. K. Sharma, and U. Ghosh, "Homomorphic encryption-based privacy-preserving federated learning in IoT-enabled healthcare system," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 5, pp. 2864–2880, Sep./Oct. 2023.

[4] S. Pandya et al., "Federated learning for smart cities: A comprehensive survey," *Sustain. Energy Technol. Assess.*, vol. 55, Feb. 2023, Art. no. 102987.

[5] J. Konečnỳ, H. B. McMahan, D. Ramage, and P. Richtárik, "Federated optimization: Distributed machine learning for on-device intelligence," 2016, *arXiv:1610.02527*.

[6] B. Zhao, K. R. Mopuri, and H. Bilen, "iDLG: Improved deep leakage from gradients," 2020, *arXiv:2001.02610*.

[7] L. Zhu, Z. Liu, and S. Han, "Deep leakage from gradients," in *Proc. 33rd Adv. Neural Inf. Process. Syst. (NIPS)*, 2019, pp. 1–11.

[8] Y. Li, Y. Zhou, A. Jolfaei, D. Yu, G. Xu, and X. Zheng, "Privacy-preserving federated learning framework based on chained secure multiparty computing," *IEEE Internet Things J.*, vol. 8, no. 8, pp. 6178–6186, Apr. 2021.

[9] S. Yang, Y. Chen, Z. Yang, B. Li, and H. Liu, "Fast secure aggregation with high dropout resilience for federated learning," *IEEE Trans. Green Commun. Netw.*, vol. 7, no. 3, pp. 1501–1514, Sep. 2023.

[10] X. Liu, H. Li, G. Xu, S. Liu, Z. Liu, and R. Lu, "PADL: Privacy-aware and asynchronous deep learning for IoT applications," *IEEE Internet Things J.*, vol. 7, no. 8, pp. 6955–6969, Aug. 2020.

[11] M. Yang, H. Cheng, F. Chen, X. Liu, M. Wang, and X. Li, "Model poisoning attack in differential privacy-based federated learning," *Inf. Sci.*, vol. 630, pp. 158–172, Jun. 2023.

[12] N. M. Hijazi, M. Aloqaily, M. Guizani, B. Ouni, and F. Karray, "Secure federated learning with fully homomorphic encryption for IoT communications," *IEEE Internet Things J.*, vol. 11, no. 3, pp. 4289–4300, Feb. 2024.

[13] X. Hao, C. Lin, W. Dong, X. Huang, and H. Xiong, "Robust and secure federated learning against hybrid attacks: A generic architecture," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 1576–1588, 2024.

[14] C. Zhang, S. Li, J. Xia, W. Wang, F. Yan, and Y. Liu, "BatchCrypt: Efficient homomorphic encryption for cross-silo federated learning," in *Proc. USENIX Annu. Tech. Conf. (USENIX ATC)*, 2020, pp. 493–506.

[15] J. Ma, S.-A. Naas, S. Sigg, and X. Lyu, "Privacy-preserving federated learning based on multi-key homomorphic encryption," *Int. J. Intell. Syst.*, vol. 37, no. 9, pp. 5880–5901, Jan. 2022.

[16] J. Park and H. Lim, "Privacy-preserving federated learning using homomorphic encryption," *Appl. Sci.*, vol. 12, no. 2, p. 734, 2022.

[17] K. Bonawitz et al., "Practical secure aggregation for privacy-preserving machine learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2017, pp. 1175–1191.

[18] J. H. Bell, K. A. Bonawitz, A. Gascón, T. Lepoint, and M. Raykova, "Secure single-server aggregation with (poly) logarithmic overhead," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2020, pp. 1253–1269.

[19] G. Xu, H. Li, S. Liu, K. Yang, and X. Lin, "VerifyNet: Secure and verifiable federated learning," *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 911–926, 2020.

[20] X. Guo et al., "VeriFL: Communication-efficient and fast verifiable aggregation for federated learning," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 1736–1751, 2021.

[21] C. Hahn, H. Kim, M. Kim, and J. Hur, "VerSA: Verifiable secure aggregation for cross-device federated learning," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 1, pp. 36–52, Jan./Feb. 2023.

[22] Y. Zhong, W. Tan, Z. Xu, S. Chen, J. Weng, and J. Weng, "WVFL: Weighted verifiable secure aggregation in federated learning," *IEEE Internet Things J.*, vol. 11, no. 11, pp. 19926–19936, Jun. 2024.

[23] S. Kadhe, N. Rajaraman, O. O. Koyluoglu, and K. Ramchandran, "FastSecAgg: Scalable secure aggregation for privacy-preserving federated learning," 2020, *arXiv:2009.11248*.

[24] L. Chen, D. Xiao, X. Xiao, and Y. Zhang, "Secure and efficient federated learning via novel authenticable multi-party computation and compressed sensing," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 10141–10156, 2024.

[25] P. Zhang, L. Xu, L. Mei, and C. Xu, "Sketch-based adaptive communication optimization in federated learning," *IEEE Trans. Comput.*, vol. 74, no. 1, pp. 170–184, Jan. 2025.

[26] C. Wu, F. Wu, Y. Cao, Y. Huang, and X. Xie, "FedGNN: Federated graph neural network for privacy-preserving recommendation," 2021, *arXiv:2102.04925*.

[27] T. Stevens, C. Skalka, C. Vincent, J. Ring, S. Clark, and J. Near, "Efficient differentially private secure aggregation for federated learning via hardness of learning with errors," in *Proc. USENIX Secur. Symp. (USENIX Secur.)*, 2022, pp. 1379–1395.

[28] S. Asoodeh, W.-N. Chen, F. P. Calmon, and A. Ozgur, "Differentially private federated learning: An information-theoretic perspective," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, 2021, pp. 344–349.

[29] A. Cheng, P. Wang, X. S. Zhang, and J. Cheng, "Differentially private federated learning with local regularization and Sparsification," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022, pp. 10122–10131.

[30] G. Yue, L. Yan, L. Kang, and C. Shen, "AdapLDP-FL: An adaptive local differential privacy for federated learning," *IEEE Trans. Mobile Comput.*, early access, Jan. 23, 2025, doi:10.1109/TMC.2025.3533090.

[31] J. Ling, J. Zheng, and J. Chen, "Efficient federated learning privacy preservation method with heterogeneous differential privacy," *Comput. Secur.*, vol. 139, Apr. 2024, Art. no. 103715.

[32] D. Wang, Y. Gao, S. Pang, C. Zhang, X. Zhang, and M. Li, "FedMPS: A robust differential privacy federated learning based on local model partition and Sparsification for heterogeneous IIoT data," *IEEE Internet Things J.*, early access, Jan. 29, 2025, doi:10.1109/JIOT.2025.3536035.

[33] L. T. Phong, Y. Aono, T. Hayashi, L. Wang, and S. Moriai, "Privacy-preserving deep learning via additively homomorphic encryption," *IEEE Trans. Inf. Forensics Security*, vol. 13, pp. 1333–1345, 2018.

[34] J. Zhang, B. Chen, S. Yu, and H. Deng, "PEFL: A privacy-enhanced federated learning scheme for big data analytics," in *Proc. IEEE Glob. Commun. Conf. (GLOBECOM)*, 2019, pp. 1–6.

Authorized licensed use limited to: Anhui University. Downloaded on March 24,2026 at 04:23:30 UTC from IEEE Xplore. Restrictions apply.

[35] Z. Yang, S. Hu, and K. Chen, "FPGA-based hardware accelerator of homomorphic encryption for efficient federated learning," 2020, *arXiv:2007.10560*.
[36] D. Stripelis et al., "Secure neuroimaging analysis using federated learning with homomorphic encryption," in *Proc. 17th Int. Symp. Med. Inf. Process. Anal. (SIPAIM)*, 2021, pp. 351–359.
[37] Y. Xu, Y. Mao, S. Li, J. Li, and X. Chen, "Privacy-preserving federal learning chain for Internet of Things," *IEEE Internet Things J.*, vol. 10, no. 20, pp. 18364–18374, Oct. 2023.
[38] J. Han and L. Yan, "Adaptive batch homomorphic encryption for joint federated learning in cross-device scenarios," *IEEE Internet Things J.*, vol. 11, no. 6, pp. 9338–9354, Mar. 2024.
[39] W. Du, M. Li, L. Wu, Y. Han, T. Zhou, and X. Yang, "A efficient and robust privacy-preserving framework for cross-device federated learning," *Complex Intell. Syst.*, vol. 9, no. 5, pp. 4923–4937, Feb. 2023.
[40] X. Yang, Z. Liu, X. Tang, R. Lu, and B. Liu, "An efficient and multi-private key secure aggregation scheme for federated learning," *IEEE Trans. Services Comput.*, vol. 17, no. 5, pp. 1998–2011, Sep./Oct. 2024.
[41] Y. Cai, W. Ding, Y. Xiao, Z. Yan, X. Liu, and Z. Wan, "SecFed: A secure and efficient federated learning based on multi-key homomorphic encryption," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 4, pp. 3817–3833, Jul./Aug. 2024.
[42] Y. Li, J. Lai, R. Zhang, and M. Sun, "Secure and efficient multi-key aggregation for federated learning," *Inf. Sci.*, vol. 654, Jan. 2024, Art. no. 119830.
[43] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *Proc. 23rd Int. Conf. Theory Appl. Cryptol. Inf. Secur. (ASIACRYPT)*, 2017, pp. 409–437.
[44] H. Chen, W. Dai, M. Kim, and Y. Song, "Efficient multi-key homomorphic encryption with packed ciphertexts with application to oblivious neural network inference," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2019, pp. 395–412.
[45] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artif. Intell. Statist. (AISTATS)*, 2017, pp. 1273–1282.
[46] C. Marcolla, V. Sucasas, M. Manzano, R. Bassoli, F. H. Fitzek, and N. Aaraj, "Survey on fully homomorphic encryption, theory, and applications," *Proc. IEEE*, vol. 110, no. 10, pp. 1572–1609, Oct. 2022.
[47] N. Koblitz, A. Menezes, and S. Vanstone, "The state of elliptic curve cryptography," *Designs, Codes Cryptogr.*, vol. 19, pp. 173–193, Mar. 2000.
[48] H. K. Maji, H. H. Nguyen, A. Paskin-Cherniavsky, T. Suad, and M. Wang, "Leakage-resilience of the Shamir secret-sharing scheme against physical-bit leakages," in *Proc. 40th Annu. Int. Conf. Theory Appl. Cryptogr. Techn. (EUROCRYPT)*, 2021, pp. 344–374.
[49] Y. Sun and Y. Yang, "Gradient privacy-preserving in federated learning via proxy re-encryption," in *Proc. 5th Int. Conf. Inf. Sci. Syst. (ICISS)*, 2022, pp. 100–106.
[50] A. Benaissa, B. Retiat, B. Cebere, and A. E. Belfedhal, "TenSEAL: A library for encrypted tensor operations using homomorphic encryption," 2021, *arXiv:2104.03152*.
[51] "Microsoft SEAL (Release 4.1)." 2023. [Online]. Available: https://github.com/Microsoft/SEAL
[52] C. Data61. "Python paillier library." 2013. [Online]. Available: https://github.com/data61/python-paillier
[53] Y. Shen et al., "Occlum: Secure and efficient multitasking inside a single enclave of Intel SGX," in *Proc. 25th Int. Conf. Archit. Support Program. Lang. Oper. Syst. (ASPLOS)*, 2020, pp. 955–970.
[54] Y. LeCun, C. Cortes, and C. J. Burges. "MNIST database." 1998. [Online]. Available: http://yann.lecun.com/exdb/mnist
[55] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms," 2017, *arXiv:1708.07747*.
[56] A. Krizhevsky, "Learning multiple layers of features from tiny images," Dept. Comput. Sci., Univ. Toronto, Toronto, ON, Canada, Rep. TR-2009, 2009.
[57] D. Anguita, A. Ghio, L. Oneto, X. Parra, and J. L. Reyes-Ortiz, "A public domain dataset for human activity recognition using smartphones," in *Proc. Esann*, 2013, p. 3.
[58] H. Yang, M. Ge, K. Xiang, and J. Li, "Using highly compressed gradients in federated learning for data reconstruction attacks," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 818–830, 2023.



Yilong Liu is currently pursuing the M.S. degree in cyberspace security with the School of Cyber Security and Computer, Hebei University, Baoding, China.

His current research interests include homomorphic encryption, federated learning, and edge computing.



Hong Zhang received the Ph.D. degree in computer science from the University of Central Florida, Orlando, FL, USA, in 2018.

He is an Assistant Professor with the School of Cyber Security and Computer, Hebei University, Baoding, China. In terms of design, he is currently working on developing the high-performance platforms of edge computing and big data. As for analysis, he focuses on improving performance, scalability, resilience, security, and load balancing of edge computing and distributed machine learning. His research interest is the design and analysis of distributed systems for edge computing and big data.



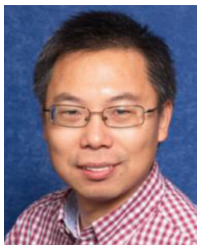
Miao Wang received the master's degree in computer science from Hebei University, Baoding, China, in 2006.

She is an Associate Professor with the School of Cyber Security and Computer, Hebei University. Her research interests are machine learning, fuzzy reasoning, and evolutionary algorithms.



Qiqi Xie (Student Member, IEEE) is currently pursuing the M.S. degree in computer technology with the School of Cyber Security and Computer, Hebei University, Baoding, China.

His current research interests include homomorphic encryption, blockchain, and federated learning.



Liqiang Wang received the Ph.D. degree in computer science from Stony Brook University, Stony Brook, NY, USA, in 2006.

He is currently a Professor with the Department of Computer Science, University of Central Florida, Orlando, FL, USA. He is the Director of Big Data Lab. He was a Faculty Member from 2006–2015 with the Department of Computer Science, University of Wyoming, Laramie, WY, USA. He was a Visiting Research Scientist with IBM T.J. Watson Research Center, Yorktown Heights, NY, USA, from 2012 to 2013. His research focuses on big data computing and analytics techniques in the following aspects: 1) improving accuracy and security of big data analysis models; 2) optimizing performance and scalability of big data processing and parallel computing systems, including multithreading, HPC, Cloud, and GPU platforms; 3) using program analysis and deep learning techniques to detect and prevent programming errors and execution anomaly in big data and/or parallel programs.

Prof. Wang received the NSF CAREER Award in 2011 and the Castagne Faculty Fellowship from 2013 to 2015.

Authorized licensed use limited to: Anhui University. Downloaded on March 24,2026 at 04:23:30 UTC from IEEE Xplore. Restrictions apply.